

Kubernetes Log Analysis

CSC 478

Matthew Potts

Vision

This Security Log Analyzer is a Kubernetes deployment that will ingest, analyze and respond to security events. Many modern cloud infrastructures generate a very large amount of security logs. Without a form of automated analysis, teams may become overwhelmed and blind to threats. This project is to create a scalable and distributed log analysis pipeline that shows how a cloud infrastructure solution can detect intrusions, prevent data breaches and maintain compliance.

This matters because security is a critical concern in cloud systems. By building this with Kubernetes, it will show not just how to detect threats but also to work on security at scale by using container orchestration, service discovery, and a persistent storage to build a resilient system that can continue running even after failure.

Architectural Overview

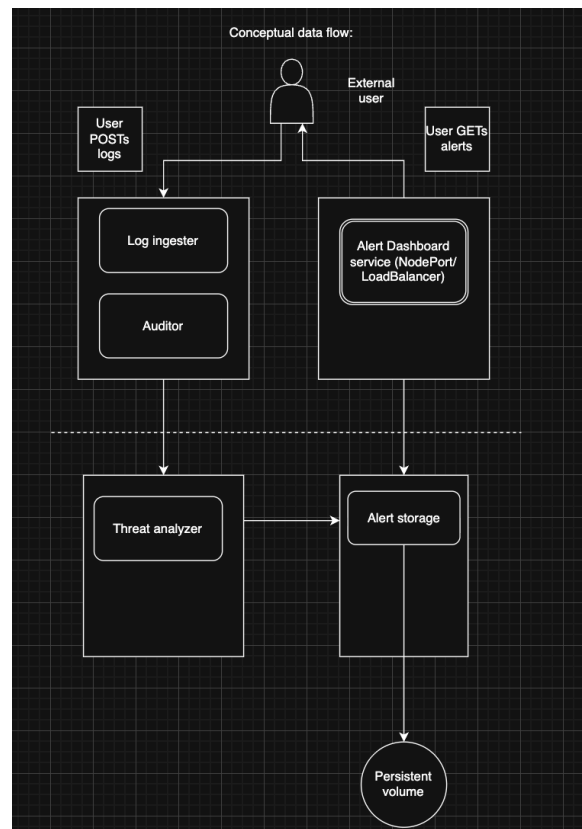


Figure 1, Conceptual data flow

Pods and service strategy: the pipeline is composed of four independent pods. The log ingestor will take incoming logs which then forwards it to the threat analyzer. The threat analyzer is a stateless pod that determines the threat of the log. Storage then persists results and allows them to be always reached. The dashboard is then the public face of the app, which allows users to view their logs at any time. The threat analyzer and the alert storage will never be exposed to the public by using ClusterIP.

Multi-container Pod: The pod for the log ingester is unique, as it contains an audit sidecar which tracks every request to the ingester. This allows for knowing who queries what and displays the sidecar pattern.

Horizontal scaling: The threat analyzer deployment will be configured to scale more than 3 replicas. This stateless threat analysis allows adding replicas and increasing the throughput without adding more complexity.

Persistence: The alert storage pod is the only stateful component as it will mount a Persistent Volume to ensure if a pod goes down, it can be spun back up, and data will not be lost.

External Facing: The log ingester and the alert dashboard are the only components with user-facing capabilities. Ideally, they will be accompanied into a single application for ease of use, where the ingester handles POST requests and the actual dashboard handles GET requests. Here NodePort and LoadBalancer will be utilized.

User Stories

Story 1: Log Ingestion – As a security analyst, I want to upload security logs from multiple sources so that I can centralize all security events in one place for analysis.

Story 2: Threat detection and alerts – As a security analyst, I want to automatically detect security threats (such as brute force attacks, SQL injections and data exfiltration attacks) so that I do not miss critical incidents and can quickly respond to attacks.

Story 3: Auditing – As a compliance officer, I want to have an immutable audit log of all log ingestions so that I can verify audit compliance that all logs are properly recorded and processed.

Risks

Risk 1: Threat rules are too broad - This could lead to many false positives and could add noise in the data. This could enable analysts to ignore real threats. A way to mitigate this is to include a confidence score onto a threat.

Risk 2: Audit sidecar slows down the ingester - logging each data point could cause the ingester to drop requests or timeout altogether. This can be mitigated by using asynchronous logging and monitoring resource usage and adding resource limits.

Risk 3: storage size – keeping logs could become slow and large at scale. In testing, we can use small data sources but in a scaled environment a real database would want to be considered.

Risk 4: aggressive scaling of the analyzer – scaling too hard on the analyzer could allow for the node to run out of memory or other resources. This can be mitigated by setting resource limits.

Test Plan

Three rules will be used for testing in this project:

The first is Brute force attempts. If more than 5 failed login attempts occur from the same IP in 60 seconds, flag as a brute force attempt.

The second is SQL injection attempts. If a request contains SQL Keywords, flag as SQL injection.

Finally, if a request size exceeds 100 MB, flag as a possible data exfiltration.

API

The API will include two external endpoints, /ingest and /alerts. /ingest will take a JSON object from a security tool into the ingester. /alerts will return a JSON object of detected threats.

Example payload for POST /ingest:

```
{
  "timestamp": date,
  "source_ip": string,
  "event_type": string,
  "user": string,
  "service": "string",
  "details": string
}
```

Example response for GET /alerts:

```
{
  "alerts": {
    "id": string,
    "rule": string,
    "severity": string,
    "source": string,
    "timestamp": date,
    "description": string
  }
}
```

Persistence Plan

What needs to be stored? Each alert needs a UUID, the rule that it violates, the severity of the threat, the source IP, the timestamp that it occurred and its description.

Where will it be stored? It will be stored directly in the PVC mounted to the Storage pod. The Persistent Volume is then backed by the local storage.